数值分析——常微分方程数值解上机报告

未央-水木 22 郁然

1 实验目的

本次上机实验的目标是使用常微分方程数值解法来计算常微分方程组的近似解：

$$\begin{cases}\dfrac{du}{dt} = -2000u(t) + 999.75v(t) + 1000.25 \\ \dfrac{dv}{dt} = u(t) - v(t)\end{cases}$$

并有初值条件$u(0) = 0, v(0) = -2$

且已给出精确解：

$$\begin{cases}u(t) = -1.499875e^{-0.5t} + 0.499875e^{-2000.5t} + 1 \\ v(t) = -2.99975e^{-0.5t} - 0.00025e^{-2000.5t} + 1\end{cases}$$

我们期望通过这次作业，求出古典四级四阶显式 Runge-Kutta 方法和隐式二级四阶 Runge-Kutta 方法计算，并与精确解比较。

2 实验方法详述

2.1 古典四级四阶显式 Runge-Kutta 方法

由所学内容，我们知道四阶显式 Runge-Kutta 方法有多种表示方法，在这里我们选取课上所说的例子，取$h = 0.001$。使用 matlab 进行计算，代码见附录 1。

2.2 隐式二级四阶 Runge-Kutta 方法

在课上已经讲过，隐式 Runge-Kutta 方法相较于显式方法可以用更低的级数来得到更高的阶数精度。但是会涉及到解方程组的问题，我们通过数值积分的思想并与迭代法结合来处理。与上类似，我们取$h = 0.001$。使用 matlab 进行计算，代码见附录 2。

3 实验结果讨论

3.1 古典四级四阶显式 Runge-Kutta 方法

利用 matlab 自带的绘图功能进行绘制，得到的结果如图 1 所示。

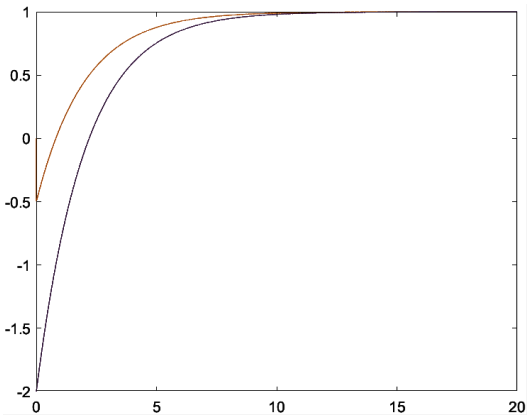


图 1 用古典四级四阶显式 Runge-Kutta 方法得到的结果

3.2 隐式二级四阶 Runge-Kutta 方法

利用 matlab 自带的绘图功能进行绘制，得到的结果如图 2 所示。

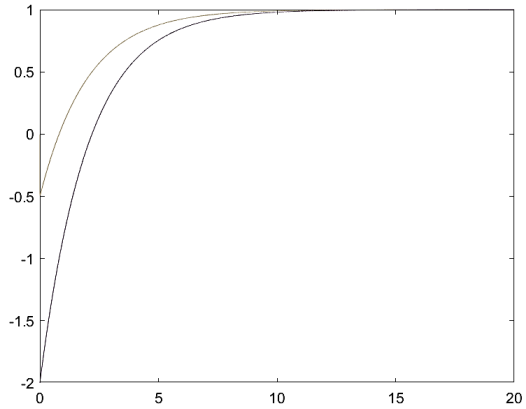


图 2 隐式二级四阶 Runge-Kutta 方法得到的结果

3.3 比较

我们通过做图发现，数值解曲线和精确解曲线拟合程度较高，在实验中如果 h 的取值过大的话，在 0 附近 u，v 导数值非常大，容易引起误差，所以在条件允许的情况下，我们应该选取较小的 h 值进行操作变换。

4 实验总结

通过这次实验我们可以看出，两者在和精确解比较时，都展现出了不错的近似效果。另外，步长的选取两者产生了较为一致的要求，这是由于它们在本质上都同属于一种思想。显式格式形式更复杂，但更直观。隐式格式虽然更为简洁，但是需要用迭代法处理，计算量较大。总之，我们应该综合考虑可用的计算资源和精度要求来分析问题并不断尝试。

另外，在探索的过程中，利用适当的计算设备和画图设备极大简化了我们研究的过程，也更好处理所得的数据，启示我们要将所学知识与适当工具结合，才能更好应对实际问题。

附录 程序代码

注：本次上机实验中，利用 MATLAB 完成相关操作，以下代码均为 MATLAB 中的代码。

1 古典四级四阶显式 Runge-Kutta 方法

```
u=@(x)(-1.499875*exp(-0.5*x)+0.499875*exp(-2000.5*x)+1);
v=@(x)(-2.99975*exp(-0.5*x)+0.00025*exp(-2000.5*x)+1);

fun=@(t,x)[-2000*x(1)+999.75*x(2)+1000.25;x(1)-x(2)];
h=0.001;
T=0:h:20;

%1.古典四级四阶显式 Runge-Kutta

U=u(T);
V=v(T);
T=T';
N=length(T);
x0=[0,-2];%初值条件

m=length(x0);
X=zeros(N,m);%函数
X(1,:)=x0;

for k=2:N
  K1=fun(T(k-1),X(k-1,:)');
  K2=fun(T(k-1)+h/2,X(k-1,:)'+h*K1/2);
  K3=fun(T(k-1)+h/2,X(k-1,:)'+h*K2/2);
  K4=fun(T(k-1)+h,X(k-1,:)'+h*K3);

  X(k,:)=X(k-1,:)+(h/6)*(K1+2*K2+2*K3+K4)';
End
plot(T,U,T,V,T,X(:,1),T,X(:,2))
```

2 隐式二级四阶 Runge-Kutta 方法

```
u=@(x)(-1.499875*exp(-0.5*x)+0.499875*exp(-2000.5*x)+1);
v=@(x)(-2.99975*exp(-0.5*x)+0.00025*exp(-2000.5*x)+1);

fun=@(t,x)[-2000*x(1)+999.75*x(2)+1000.25;x(1)-x(2)];
h=0.001;
T=0:h:20;
U=u(T);
```

```
V=v(T);
T=T';
N=length(T);

%2.隐式二级四阶 Runge-Kutta
N=length(T);
x0=[0,-2];%初值条件

m=length(x0);
X=zeros(N,m);%函数
X(1,:)=x0;

K_1 = zeros(m,N);
K_2 = zeros(m,N);
for k=2:N
    for j=1:50%隐式格式需要迭代求解
        K1=K_1(:,k-1);
        K2=K_2(:,k-1);
        K_1(:,k-1)=fun(T(k-1)+(0.5-(3^(0.5))/6)*h,X(k-
1,:)'+0.25*h*K_1(:,k-1)+(0.25-(3^(0.5))/6)*h*K_2(:,k-1));
        K_2(:,k-1)=fun(T(k-1)+(0.5+(3^(0.5))/6)*h,X(k-
1,:)'+0.25*h*K_2(:,k-1)+(0.25+(3^(0.5))/6)*h*K_1(:,k-1));
        if norm(K_1(:,k-1)-K1)<1e-12 & norm(K_2(:,k-1)-K2)<1e-10
            K_1(:,k) =K_1(:,k-1);K_2(:,k) =K_2(:,k-1);
            break
        end
        K_1(:,k) =K_1(:,k-1);K_2(:,k) =K_2(:,k-1);
    end
X(k,:) =X(k-1,:)+(h/2)*(K_1(:,k)+K_2(:,k))';
end

plot(T,U,T,V,T,X(:,1),T,X(:,2))
```